神经认知诊断方法复现及其改进

摘要

学生表现预测是智慧教育中的一个基本问题。在客观题领域，学生作答结果只有正确和错误两种情况，因此，学生表现预测问题可以归结为二分类问题。教育数据挖掘领域中预测学生客观题表现的方法主要有认知诊断和矩阵分解两大类。其中，现有的认知诊断方法通常通过手动设计的函数表征学生参数和试题参数之间的交互函数，不足以捕捉学生和试题之间的复杂关系。针对这一问题，Liu 提出了一个通用的神经认知诊断模型(NeuralCDM)，该模型结合了神经网络来学习复杂的学生-试题交互函数，以获得准确和可解释的诊断结果。具体来说，NeuralCDM 将学生和习题特征投射到特征向量，并利用多层神经网络对其进行建模。其中单调性假设应用于确保这两个因素的可解释性。在两个真实数据集上显示了 NeuralCD 的有效性具有准确性和可解释性的框架。

本课程报告采用 python 语言和 pytorch 框架实现了认知诊断模型，复现了认知诊断模型在公共数据集上预测表现，并跟其他传统模型进行了对比；此外，本文采用 python 爬虫的方法收集了 2018 年秋季学期湖南大学《信号与系统》慕课课程的客观题作答数据，制作了 HNU_SYS 数据集，并且在 HNU_SYS 数据集复现了神经认知诊断模型；最后，针对于概率矩阵分解和认知诊断模型以及神经认知诊断模型侧重于提取数据的不同特征这一特点，本文提出了一种嵌入注意力机制网络的融合模型，在决策层融合三类模型，实验结果表明，融合模型相对于 3 类独立模型(NeuralCDM,HO-DINA,PMF)在 HNU_SYS 数据集上的预测效果分别提升了 1.9%，5.2%，4.7%。

1 研究意义及研究现状

大数据时代的到来，颠覆了传统的校园面授教学模式。智慧教育和在线教育的理念逐渐深入人心，以学习者为中心的个性化学习成为近年来教育数据挖掘领域的热点问题。在学习者自适应的学习过程中，智能教学系统预测学生的答题表现，帮助学生提高、巩固所学的知识，从而完善学生的个性化知识体系结构，达到因材施教的目的。因此，如何针对特定学习者的知识结构、能力水平，预测学生的答题表现，是在线智慧教育的核心问题。然而，影响学习者作答表现的因素种类繁杂，学习者很难准确判断自己在某一学习阶段中的认知状态，因此，亟需找到一种成绩预测方法，帮助学习者高效学习。

矩阵分解是推荐系统领域中协同过滤方法的一种。因此，除了矩阵分解，常

用的推荐系统方法都可以适用于成绩预测。以推荐系统的视角分析，试题推荐系统的用户对应于学习者，推荐的物品对应于试题，推荐系统的历史用户-物品交互记录对应于学生的试题作答记录。试题作答记录包括主观题和客观题的记录，客观题通常用二元离散变量来标记作答正确与否，主观题的得分记录归一化为 0-1 之间的连续值。近年来，国内外学者将推荐系统的建模思想应用到学生成绩预测中，取得了良好的预测效果，试题推荐系统的可解释性使得预测的结果列表更真实可信。

目前，推荐系统的主流方法可分为 3 类，分别是基于内容的推荐，基于社交网络的推荐，基于协同过滤的推荐。基于内容和推荐系统利用系统用户已评分的物品作为训练，物品由一系列特征属性来刻画，推荐过程与用户的相似程度无关；基于社交网络的推荐系统能利用用户的社交线索来推荐物品，用户之间存在信任的反馈机制，使得推荐结果更具鲁棒性；基于协同过滤的推荐是推荐系统中最常用，最普遍的方法，基本思想是通过已知的评分估计未知的评分，它通过对大量用户给出的评分做出协同处理，并给出推荐结果，大多数情况下，用作训练的评分矩阵是稀疏的。因此在大数据的情况下，协同过滤推荐结果的精度难以保证。

此外，协同过滤推荐方法又可以分为 3 类：基于用户的协同过滤推荐、基于物品的协同过滤推荐、基于模型的协同过滤推荐。基于用户和物品的协同过滤，其核心思想是根据历史评分记录计算用户之间、物品之间的相似度，常用的相似度计算方法有余弦相似度、皮埃尔相似度，显然，选取的相似度指标的好坏决定了推荐系统的性能，除了人工设计的相似度指标，还可以从评分数据学习到相似度的计算，经过机器学习算法训练，得到相似度矩阵，基于模型的协同过滤推荐算法，其代表是概率矩阵分解算法 PMF，又称潜在因子模型 LPM，PMF 因其算法复杂度低，易于实现和部署的优点，在规模较小的数据集上有较高的预测性能。PMF 最初应用在矩阵补全的任务上，它将历史评分矩阵分解为低维度的用户矩阵和物品矩阵，通过不断地迭代，学习用户和物品的潜在特征向量，训练结束后，预测用户对试题的评分，用户特征向量与物品特性向量的点积结果，作为用户对试题的预测得分。然而，因其训练时需要扫描所有的用户行为记录，所以 PMF 很难实现实时的推荐，且受到稀疏数据的限制，PMF 在大数据背景下没能取得预期的表现。

认知诊断方法也可用于学生的成绩预测，同时可以诊断出学生对于试题包含的各个知识点的掌握程度。认知诊断方法首先通过各种不同的模型对学生进行能力建模，建模所得到的参数均有很强的解释性意义，并用于学生得分预测。在认知诊断模型中，试题受到一组显示知识点所关联，及试题会考察一组给定的，相关联的知识点，试题-知识点关联矩阵被称为 Q-矩阵，认知诊断模型需要在建

模学生的同时保证较好的可解释性，常见的认知诊断模型有项目反应理论(IRT)和 DINA 模型。IRT 将学生描述成在一维的能力值，并结合试题参数(难度、区分度)等对学生进行建模。DINA 模型结合 Q 矩阵，将学生描述成一个在多维知识点上的掌握向量，从而对学生建模。

2 相关工作

2.1 问题定义及准备知识

在习题推荐场景中，有 N 个学生，M 个习题，K 个知识概念，可以表示为 $S = \{s_1, s_2,\ldots,s_N\}$， $E = \{E_1, e_2,\ldots,e_M\}$和 $K_n= \{k_1, k_2, \ldots k_K\}$。每个学生将选择一些习题练习,得到试题作答记录 $R=\{R_{ij}\}_{N\times M}$，R 用三元组数据(s、e、r) 表示,其中 $e \in E$，$s\in S$，r 是学生 s 在习题 e 上的作答得分。此外,我们还需要专家标注的 Q-矩阵(试题-知识点关联矩阵)，$Q = \{Q_{ij}\}_{M\times K}$,其中，如果习题 e_i 与知识概念 k_j 有关，则 $Q_{ij}= 1$，否则 $Q_{ij}= 0$。基于学生的作答记录 R 和 Q 矩阵，成绩预测系统的目标是预测学生客观题作答结果是正确还是错误。

2.2 矩阵分解

近年来,国内外学者证明了矩阵分解方法在矩阵补全任务中的可行性。因此，矩阵分解方法开始被应用于试题推荐系统中。XX 等人提出了奇异值分解，拉开了矩阵分解研究的帷幕，SVD 将矩阵分解为 3 个矩阵的乘积，左右两个矩阵分别表示用户/项目隐含因子矩阵，中间矩阵为奇异值矩阵并且是对角矩阵，每个元素满足非负性，并且逐渐减小。然而 SVD 要求矩阵是稠密的，即矩阵里的元素要非空，否则就不能运用 SVD 分解，显然 SVD 不满足矩阵补全任务的要求。针对这一问题，Simon Funk 提出了 FunkSVD 的方法。它不在将矩阵分解为 3 个矩阵，而是分解为 2 个低秩的用户项目矩阵，同时降低了计算复杂度：

$$\min_{q^*,p^*} \sum_{(u,i)\in k} (r_{ui} - q_i^T p_u)^2 \tag{1}$$

显然，FunkSVD 将矩阵分解转变为一个最优化问题，,它借鉴线性回归的处理方法，优化目标为最小化训练数据的均方误差，其优化的目标函数为公式()，在优化目标函数的同时，寻求最佳的用户潜在向量和项目潜在向量。为了防止过拟合，目标函数通常加上带有 L2 正则化的项：

$$\min_{q^*,p^*} \sum_{(u,i)\in k} (r_{ui} - q_i^T p_u)^2 + \lambda(\| q_i \|^2 + \| p_u \|^2) \tag{2}$$

概率矩阵分解是 FunkSVD 的概率解释，在试题推荐场景中，PMF 假定试题作答记录中的元素 $R_{i,j}$ 是由学生潜在特征向量和试题潜在特征向量的内积决定，期中，学生潜在特征向量和试题潜在特征向量均服从均值为 0，方差为 $\sigma_U^2 I, \sigma_V^2 I$ 的高斯正态分布：

$$p(V \mid \sigma_V^2) = \prod_{j=1}^{M} N(V_j \mid 0, \sigma_V^2 I) \tag{3}$$

$$p(V \mid \sigma_V^2) = \prod_{j=1}^{M} N(V_j \mid 0, \sigma_V^2 I) \tag{4}$$

因此，学生作答得分也服从高斯分布，即 $R_{i,j} \sim N(U_i^T V_j, \sigma^2)$ 。

由贝叶斯公式后验概率分布公式，可以得到潜在变量 U,V 的对数后验概率分布为：

$$\begin{aligned}\ln p(U,V \mid R) = &-\frac{1}{2\sigma^2}\sum_{i=1}^{N}\sum_{j=1}^{M} I(R_{i,j} - U_i^T V_j) - \frac{1}{2\sigma_U^2}\sum_{i=1}^{N} U_i^T U_i - \frac{1}{2\sigma_V^2}\sum_{j=1}^{M} V_j^T V_j \\ &-\frac{1}{2}((\sum_{i=1}^{N}\sum_{j=1}^{M} I_{ij})\ln\sigma^2 + ND\ln\sigma_U^2 + MD\ln\sigma_V^2) + C\end{aligned} \tag{5}$$

2.3HO-DINA 模型

HO-DINA 模型是在 DINA 模型的基础上发展起来的。DINA 模型属于离散型的认知诊断模型，其学生对各知识点的掌握状态只有掌握和未掌握两种状态。DINA 模型考虑了学习者的多维知识水平与试题的多维特性来建模学习者的试题作答结果预测。DINA 模型的描述：

$$P(r_{ij} = 1 \mid \delta_{ij}, s_j, g_j) = (1 - s_j)^{\delta_{ij}} g_j^{(1-\delta_{ij})} \tag{6}$$

$$\delta_{ij} = \prod_{k=1}^{K} \alpha_{ik}^{q_{jk}} \tag{7}$$

上式中，δ_{ij} 表示学习者 i 是否掌握试题 j 。s_j ，g_j 分别为试题 j 的粗心参数与猜测参数。s_j=$P(r_{ij} = 0 \mid \delta_{ij} = 1)$ ，表示学习者 i 在掌握试题 j 的情况下，因粗心导致错误作答的概率，g_j=$P(r_{ij} = 1 \mid \delta_{ij} = 0)$ 表示学习者 i 在未掌握试题 j 的情况下，通过猜测行为正确作答的概率。

上式中，q_{jk} 表示试题 j 是否考察知识点 k 。元素 q_{jk} 构成认知诊断模型中的试题-知识点考察矩阵 $\boldsymbol{Q}$ 。δ_{ij} 、α_{ik} 、q_{jk} 取值为 0 或 1 的二元类型，即只有掌握和未掌握或考察和未考察两种状态。

为了更精准的描述试题掌握程度 δ_{ij} ，DINA 模型还有一种变型结构，如下式，δ_{ij} 取值范围为 0-1 之间的连续值。

$$P(r_{ij} = 1 \mid \delta_{ij}, s_j, g_j) = (1 - s_j)\delta_{ij} + g_j(1 - \delta_{ij}) \tag{8}$$

HO-DINA 模型在 DINA 模型基础上，考虑学习者的知识水平与其自身能力特征存在式（2）中的作用关系，但是其忽略了学习者的努力特征同样作用于知识水平。另外，HO-DINA 模型中，知识水平 α_{ik} 取值同样为二元类型。

3 本文方法

NeuralCD 模型使用一个四层的神经网络从数据中学习学习者与试题之间的交互关系。神经网络的输入为答题记录中学习者和试题的 one-hot 向量 $\boldsymbol{x}^s$ 和 $\boldsymbol{x}^e$，输出为学习者答对该试题的预测概率 y。

神经网络的第一层表示为：

$$\boldsymbol{x} = Q_e \circ (\boldsymbol{h}^s - \boldsymbol{h}^{diff}) \times h^{disc} \tag{9}$$

式(9)中，$\boldsymbol{h}^s$ 为学习者的知识水平向量，表示学习者对每个知识点的掌握程度，$\boldsymbol{h}^s = sigmoid(\boldsymbol{x}^s \times A)$，sigmoid 为 sigmoid 函数，$A$ 为神经网络的训练参数矩阵。$\boldsymbol{h}^{diff}$ 为试题的知识点难度向量，表示试题对每个知识点考察的难度，$\boldsymbol{h}^{diff} = sigmoid(\boldsymbol{x}^e \times B)$，$B$ 为神经网络的训练参数矩阵。h^{disc} 为试题的区分度参数，$\boldsymbol{h}^{disc} = sigmoid(\boldsymbol{x}^e \times D)$，$D$ 为神经网络的训练参数矩阵。$\boldsymbol{Q}_e$ 为试题与知识点的关联向量，$\boldsymbol{Q}_e = x^e \times \boldsymbol{Q}$。$\boldsymbol{x}$ 为该层的输出向量。神经网络的第二、三、四层均为全连接层，并使用 sigmoid 函数作为激活函数。

$$\boldsymbol{f}_1 = sigmoid(\boldsymbol{W}_1 \times \boldsymbol{x}^T + \boldsymbol{b}_1) \tag{10}$$

$$\boldsymbol{f}_2 = sigmoid(\boldsymbol{W}_2 \times \boldsymbol{f}_1 + \boldsymbol{b}_2) \tag{11}$$

$$y = sigmoid(\boldsymbol{W}_3 \times \boldsymbol{f}_2 + \boldsymbol{b}_3) \tag{12}$$

式(10~12)中，$\boldsymbol{W}_1$、$\boldsymbol{W}_2$、$\boldsymbol{W}_3$、$\boldsymbol{b}_1$、$\boldsymbol{b}_2$、$\boldsymbol{b}_3$ 为各网络层中的权重参数与偏置参数，$\boldsymbol{f}_1$、$\boldsymbol{f}_2$、y 分别为各网络层的输出参数。

NeuralCDM 模型的结构示意图如下：

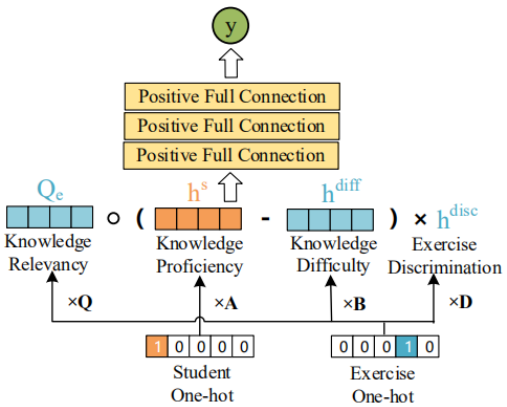


图 1：NeuralCDM 模型结构图

与上面提到的认知诊断模型基于统计学理论的求解算法不同，NeuralCD 模型采用梯度下降法对参数进行求解，在求解过程中缺少对现有先验信息的使用，例如 HO-DINA 模型在求解过程中会结合学习者某一维度的知识水平服从正态分布这一先验信息，而 NeuralCD 则没有考虑这一点。因而 NeuralCD 模型对小样本的评估效果不理想，且评估结果缺乏说服力。

4 复现实验结果及分析

4.1 实验环境的搭建

复现 Neural 模型需要安装 python3.6 版本，pytorch>=1.0 版本。此外，还需要安装 numpy、json、sklearn 等依赖包。为了复现需要，本地 PC 端安装了 Anaconda3.6 版本。Anaconda3.6 版本是一个 python 个版本的管理软件，可以本地 PC 端兼容各种不同的虚拟环境。安装好 anaconda 后，在 cmd 命令行输入命令“conda env list” 可查看本地安装的虚拟环境。

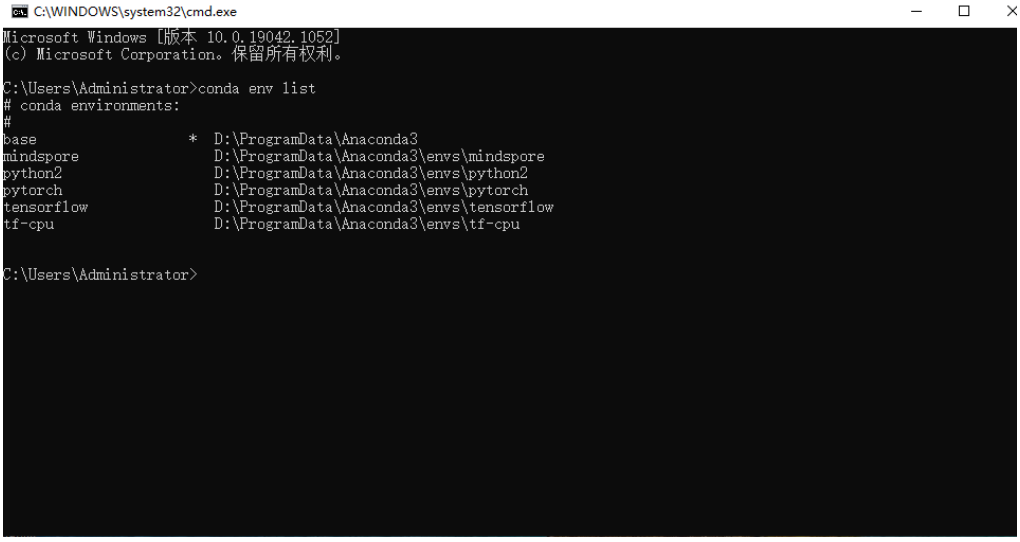


图 2：查看本地的虚拟环境

为了运行用 pytorch 框架编写的 python 代码，我们需要安装含有 pytorch 框架的虚拟环境。因此，我们需要重新创建一个虚拟环境，用来安装 pytorch-CPU 版本或者 pytorch-GPU 版本。因为本地 PC 机的显卡不是英伟达公司的，所以选择安装 pytorch-CPU 版本。cmd 命令行输入“conda create -n pytorch21 python3.6”，创建一个 python 版本为 3.6 的、名为 pytorch21 的环境变量。

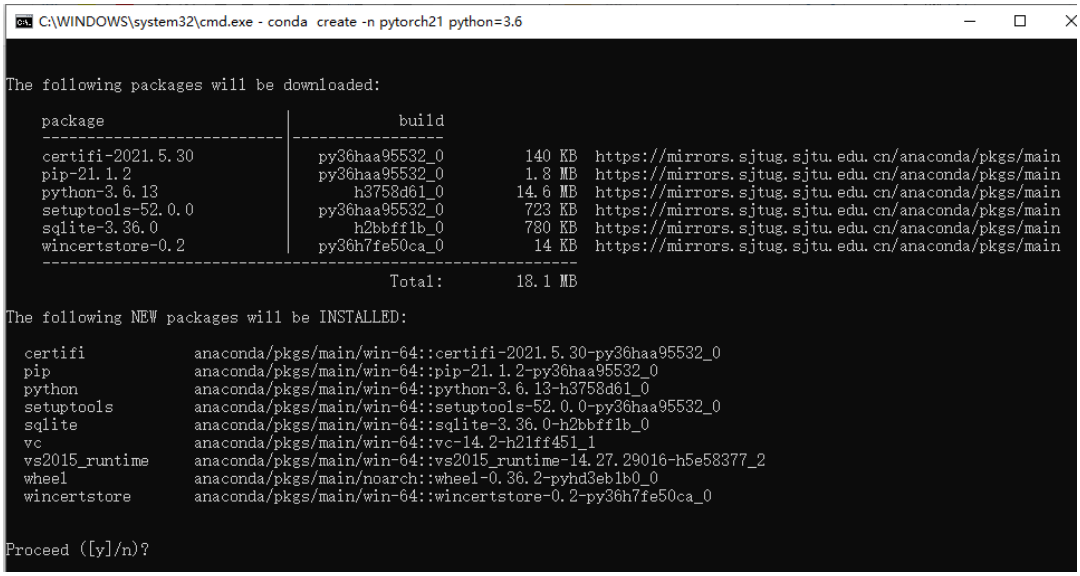


图 3：创建一个新的虚拟环境

输入 y，继续安装，再次检查本地安装的虚拟环境，可以看到本地已经创建 pytorch21 环境变量。

```
C:\Users\Administrator>conda env list
# conda environments:
#
base                  *  D:\ProgramData\Anaconda3
mindspore                D:\ProgramData\Anaconda3\envs\mindspore
python2                  D:\ProgramData\Anaconda3\envs\python2
pytorch                  D:\ProgramData\Anaconda3\envs\pytorch
pytorch21                D:\ProgramData\Anaconda3\envs\pytorch21
tensorflow               D:\ProgramData\Anaconda3\envs\tensorflow
tf-cpu                   D:\ProgramData\Anaconda3\envs\tf-cpu
```

图 4：查看已创建的环境变量

输入命令“activate pytorch21”进入到虚拟环境 pytorch21 中，再输入命令“pip install http://download.pytorch.org/whl/cpu/torch-0.4.0-cp36-cp36m-win_amd64.whl ”“pip install torchvision”安装 pytorch-CPU 版本

```
(pytorch21) C:\Users\Administrator>pip install http://download.pytorch.org/whl/cpu/torch-0.4.0-cp36-cp36m-win_amd64.whl
Collecting torch==0.4.0
  Downloading http://download.pytorch.org/whl/cpu/torch-0.4.0-cp36-cp36m-win_amd64.whl (31.8 MB)
     |████████████████████████████████| 31.8 MB 6.4 MB/s
Installing collected packages: torch
Successfully installed torch-0.4.0

(pytorch21) C:\Users\Administrator>pip install torchvision
Collecting torchvision
  Downloading torchvision-0.10.0-cp36-cp36m-win_amd64.whl (920 kB)
     |████████████████████████████████| 920 kB 731 kB/s
Collecting numpy
  Downloading numpy-1.19.5-cp36-cp36m-win_amd64.whl (13.2 MB)
     |████████████████████████████████| 13.2 MB 6.4 MB/s
Collecting pillow>=5.3.0
  Downloading Pillow-8.2.0-cp36-cp36m-win_amd64.whl (2.2 MB)
     |████████████████████████████████| 2.2 MB 6.4 MB/s
Collecting torch==1.9.0
  Downloading torch-1.9.0-cp36-cp36m-win_amd64.whl (222.0 MB)
     |████████████████████████████████| 222.0 MB 47 kB/s
Collecting dataclasses
  Downloading dataclasses-0.8-py3-none-any.whl (19 kB)
Collecting typing-extensions
  Downloading typing_extensions-3.10.0.0-py3-none-any.whl (26 kB)
Installing collected packages: typing-extensions, dataclasses, torch, pillow, numpy, torchvision
  Attempting uninstall: torch
    Found existing installation: torch 0.4.0
    Uninstalling torch-0.4.0:
      Successfully uninstalled torch-0.4.0
Successfully installed dataclasses-0.8 numpy-1.19.5 pillow-8.2.0 torch-1.9.0 torchvision-0.10.0 typing-extensions-3.10.0.0

(pytorch21) C:\Users\Administrator>
```

图 5：安装 pytorch-CPU 版本

进入 python 命令行，输入 “import torch”、“print(torch.__version__)”可检验 pytorch 是否安装成功，由下图可知，本地成功安装 pytorchCPU1.9 版本。

```
(pytorch21) C:\Users\Administrator>python
Python 3.6.13 |Anaconda, Inc.| (default, Mar 16 2021, 11:37:27) [MSC v.1916 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> import torch
>>> print(torch.__version__)
1.9.0+cpu
>>>
```

图 6：检查 pytorch 是否安装成功

4.2 代码结构分析

```
C:.
|   .gitignore
|   a.txt
|   config.txt
|   data_loader.py
|   divide_data.py
|   Fei-Wang-AAAI2020.pdf
|   model.py
|   predict.py
|   README.md
|   train.py
|
├---.idea
|   |   .gitignore
|   |   misc.xml
|   |   modules.xml
|   |   Neural_Cognitive_Diagnosis-NeuralCD-master.iml
|   |   workspace.xml
|
├---data
|   |   log_data.json
|   |   test_set.json
|   |   train_set.json
|   |   train_slice.json
|   |   val_set.json
|
├---model
|   |   .gitkeep
|   |   model_epoch1
|   |   model_epoch2
|   |   model_epoch3
|
├---result
|   |   .gitkeep
|   |   model_val.txt
|
└---__pycache__
    |   data_loader.cpython-37.pyc
    |   model.cpython-37.pyc
```

图 7：代码文件结构

主程序文件有 data_loader.py、divide_data.py、 model.py 、predict.py 、train.py 五个。data_loader.py 文件主要负责导入数据，对数据进行预处理，将原始数据集转变为适合模型处理的数据；divide_data.py 文件负责划分训练集、测试集、验证集；model.py 文件负责定义神经认知诊断模型，运用 pytorch 框架实现神经网络全连接层；train.py 文件负责调用训练集，利用优化算法训练模型结构参数；predict.py 负责调用测试集，检验模型的泛化能力。

除了 5 个主程序文件之外，data、model、result 等三个文件夹也十分重要。其中，data 文件夹负责存放 json 格式的训练集数据，测试集数据、验证集数据；model 文件夹保存了每一个训练轮次下的模型；result 文件夹保存了模型训练过程中的中间结果，例如学生对各个知识点的掌握程度。

4.3 在 ASSIT2009-2010 数据集上复现

ASSIT2009-2010 属于公开数据集，常用于成绩预测和知识追踪算法。此外，我们通过 python 爬取了 2018 年秋季学期湖南大学《信号与系统》慕课课程数据，构建了 HNU_SYS 数据集。关于 ASSIT2009-2010 和 HNU_SYS 数据集的信息如下表所示：

表 1：数据集介绍

数据集	ASSIT2009-2010	HNU_SYS
学生数	4163	466
习题数	17746	90
知识点数	123	6

运行 Python 代码的集成开发环境选择的是 Pycharm。在实验环境的搭建上，我们已经在 anaconda 下创建了一个包含 pytorchCPU 框架的虚拟环境 pytorch21，在使用 Pycharm 运行 NeuralCDM 代码时，需要配置 python 解释器为 pytorch21。在“settings”中选中“python interpreter”选中 pytorch 即可。

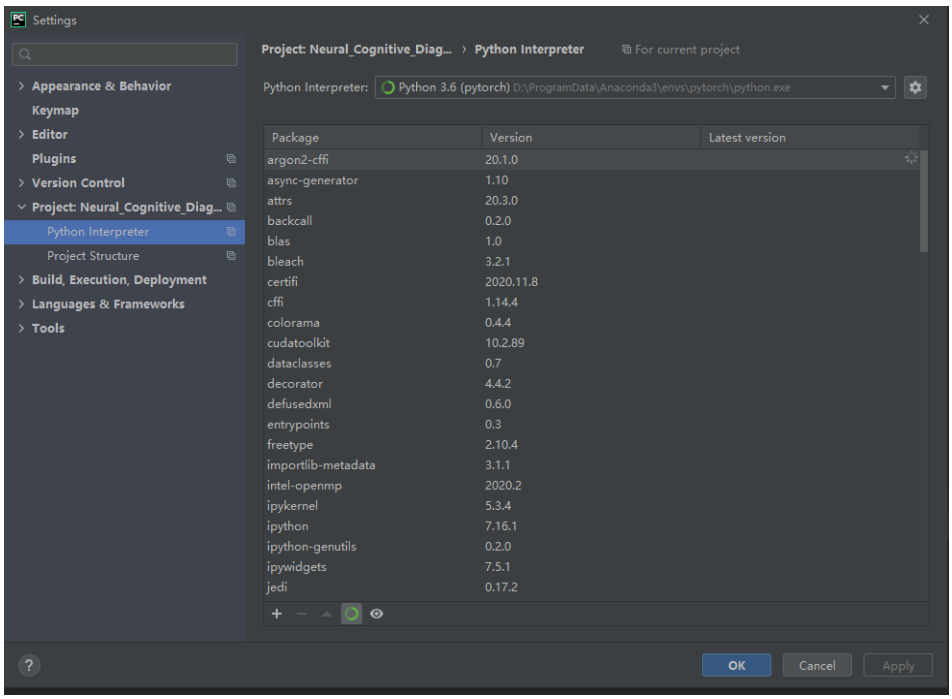


图 8：Pycharm 中配置 python 解释器

配置好 python 解释器后，下一步即开始 NeuralCDM 模型的训练，数据集 ASSIT2009-2010 的 json 格式已经被事先存储在 data 文件夹下，分别是 train_set.json、 test_set.json 、val_set.json。接着选中 train.py 文件，配置运行参数“CPU 5”，表示使用 CPU 训练 5 个轮次，训练完毕后，可见 model 文件夹下保存了每个轮次训练模型。

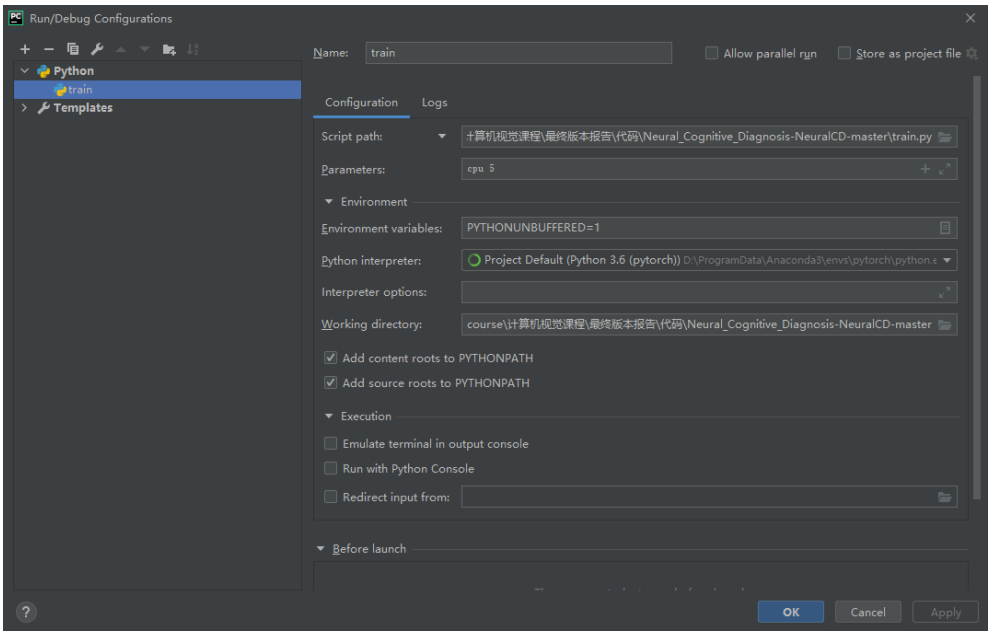
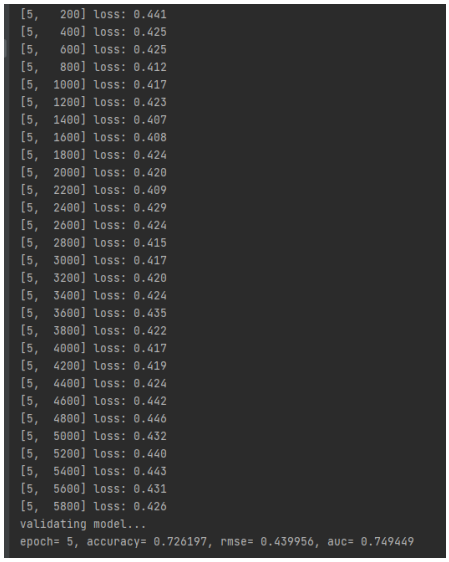


图 9：train.py 配置运行参数

以第五个训练轮次为例，训练结果如下图所示。由下图可知，第五个训练轮次的拟合精度为 0.726197，RMSE 值为 0.439956。



```
[5,   200] loss: 0.441
[5,   400] loss: 0.425
[5,   600] loss: 0.425
[5,   800] loss: 0.412
[5,  1000] loss: 0.417
[5,  1200] loss: 0.423
[5,  1400] loss: 0.407
[5,  1600] loss: 0.408
[5,  1800] loss: 0.424
[5,  2000] loss: 0.420
[5,  2200] loss: 0.409
[5,  2400] loss: 0.429
[5,  2600] loss: 0.424
[5,  2800] loss: 0.415
[5,  3000] loss: 0.417
[5,  3200] loss: 0.420
[5,  3400] loss: 0.424
[5,  3600] loss: 0.435
[5,  3800] loss: 0.422
[5,  4000] loss: 0.417
[5,  4200] loss: 0.419
[5,  4400] loss: 0.424
[5,  4600] loss: 0.442
[5,  4800] loss: 0.446
[5,  5000] loss: 0.432
[5,  5200] loss: 0.440
[5,  5400] loss: 0.443
[5,  5600] loss: 0.431
[5,  5800] loss: 0.426
validating model...
epoch= 5, accuracy= 0.726197, rmse= 0.439956, auc= 0.749449
```

图 10：训练过程截图

最后是采用训练后的模型预测在测试集上的表现，此时需选择 predict.py 文件，配置运行参数为“5”，代表采用第 5 个训练轮次得到的模型进行预测。运行结果如下：

```
predict
D:\ProgramData\Anaconda3\envs\pytorch\python.exe C:/Users/Administrator/Desktop/course/计算机视觉课程/最终版本报告/代码/Neural_Cognitive_Diagnosis-NeuralCD-master/predict.py 5
testing model...
epoch= 5, accuracy= 0.728156, rmse= 0.439329, auc= 0.751993

Process finished with exit code 0
```

图 11：第 5 个训练轮次的预测结果

在 70%训练集下，神经认知诊断模型(NeuralCDM)与其他传统模型在公开数据集上的表现见下表

表 2：公开数据集下 NeuralCDM 与其他传统模型的性能比较

	Math			ASSIST		
Model	Accuracy	RMSE	AUC	Accuracy	RMSE	AUC
DINA	0.593±.001	0.487±.001	0.686±.001	0.650±.001	0.467±.001	0.676±.002
IRT	0.782±.002	0.387±.001	0.795±.001	0.674±.002	0.464±.002	0.685±.001
MIRT	0.793±.001	0.378±.002	0.813±.002	0.701±.002	0.461±.001	0.719±.001
PMF	0.763±.001	0.407±.001	0.792±.002	0.661±.002	0.476±.001	0.732±.001
NeuralCDM	0.792±.002	0.378±.001	0.820±.001	**0.719±.008**	**0.439±.002**	**0.749±.001**
NeuralCDM+	**0.804±.001**	**0.371±.002**	**0.835±.002**	-	-	-

4.4 在 HNU_SYS 数据集上的复现

HNU_SYS 原始数据集格式为 txt 格式，为了转变为适合于 python 代码处理的 json 格式，需要采用 MATLAB 脚本文件将 txt 格式转变为 json 格式。MATLAB 脚本文件命名为 data2json。

数据转换完成后，NeuralCDM 在 HNU_SYS 数据集上的复现步骤与上一小节一致：首先，更改 config.txt 文件，将 HNU_SYS 数据集的学生数、习题数、知识点数对应更改为 466,90,6。

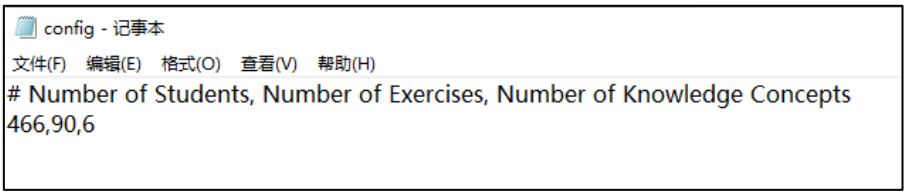


图 12：修改 config.txt

然后，选择 train.py 文件，运行参数设置为“CPU 100”，即采用 CPU 训练 100 个轮次。以第 98-100 轮的训练为例，训练过程如图所示

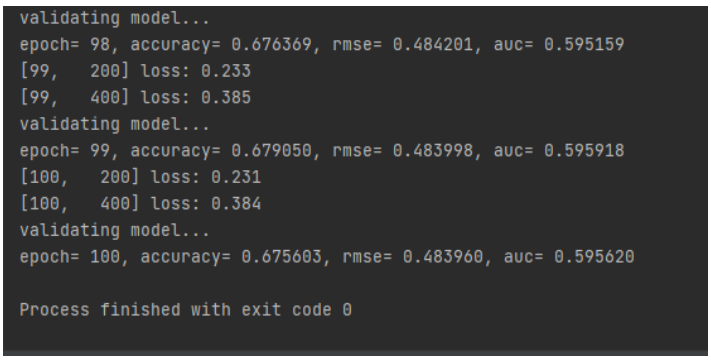


图 13：HNU_SYS 数据集下的训练过程

最后选择 predict.py 文件，运行参数设置为“100”即采用第 100 轮训练的模型预测在测试集上的表现。运行结果如下：

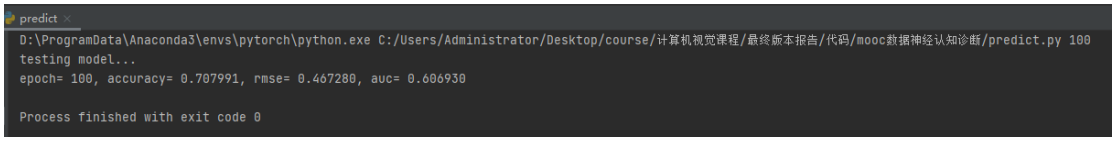


图 14：HNU_SYS 数据集下第 100 轮训练模型的预测结果

在 70%训练集下，神经认知诊断模型(NeuralCDM)与其他传统模型在 HNU_SYS 数据集上的表现见下表:

表 3：HNU_SYS 数据集下 NeuralCDM 与其他传统模型的比较

Model	HNU_SYS		
	Accuracy	RAE	RMSE
PMF	0.7786	0.2369	0.4856
HO-DINA	0.7731	0.2258	0.4751
Fuzzy-CDF	0.7350	0.2210	0.4702
DINA	0.6715	0.2763	0.5257
NeuralCDM	0.8059	0.2012	0.4589

5 神经认知诊断模型改进方法

目前学生成绩预测的研究主要存在以下 2 个方面的问题：

(1)如何充分挖掘历史评分记录的信息，兼顾矩阵分解方法的共性特征和认知诊断方法的个性特征，并实现学习者个性化的权重参数学习？

(2)如何使得成绩预测算法能够适应不同规模的数据集，提高模型的泛化能力？

为了解决上述问题，本文拟在 NeuralCDM 模型的基础上做出改进，提出一种集成分类方法，用于实现成绩预测模型的决策层级融合。近年来，有学者将注意力机制引入到习题之间相似度的学习，通过级联一层神经网络，从而学习到试题之间的相似度信息。本文拟将注意力机制网络应用于模型的融合，融合的模型涵盖协同过滤和认知诊断两大主流成绩预测方法，由于协同过滤和认知诊断从历史评分数据中学习到的特征信息具有很大的差异性，因此，经过层级融合之后的模型，能够结合各自模型的优势并弥补模型的不足，保证生成的习题推荐列表的准确性和可解释性。本文提出的嵌入注意力机制的成绩预测方法步骤如下：

本文方法的主要创新点如下：

(1)本文首创通过注意力机制网络在决策层融合协同过滤模型和认知诊断模型，有效利用了二者模型的决策优势，提升成绩预测的精度，并增强模型的可解释性。

(2)该方法能够针对不同学习者，学习到不同的模型权重因子，学习个性突出的学生，在认知诊断方法上获得得权重高，学习共性强的学生，在协同过滤方法上获得得权重高，从而实现了精准的成绩预测。

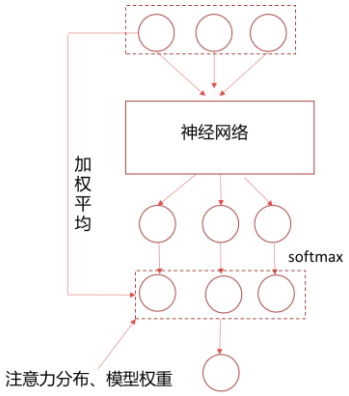


图 15：改进的集成分类模型结构示意图

6 改进的实验结果及分析

以融合 NeuralCDM、HO-DINA、PMF 三个模型为例，说明改进的集成分类模型的具体步骤：

(1)各自模型在同一训练集下训练，得到关于所有数据样本的预测概率向量。

(2)若融合 NeuralCDM、HO-DINA、PMF3 个模型，首先分别计算 3 个模型对第一个数据的预测值。

(3)三个模型的预测值作为带注意力机制的神经网络的输入，输出为第一个样本的融合预测值。

(4)采用交叉熵损失函数计算 loss。

(5)梯度下降优化，得到最佳的注意力分布值，即模型权重值。

(6)利用训练得到模型权重，预测测试集上的表现。

改进的集成分类方法和各个子分类模型的训练过程互不干扰，属于独立的融合阶段，因此，该方法具有很好的可扩展性。集成分类方法的实现采用 MATLAB 代码编写，使用到了神经网络工具箱。

在 HNU_SYS 数据集下，70%训练比例得到的集成模型与其他模型的性能比较见下表：

Model	HNU_SYS		
	Accuracy	RAE	RMSE
PMF	0.7786	0.2369	0.4856
HO-DINA	0.7731	0.2258	0.4751
Fuzzy-CDF	0.7350	0.2210	0.4702
DINA	0.6715	0.2763	0.5257
NeuralCDM	0.8059	0.2012	0.4589
融合方法	0.8256	0.1896	0.4456

7 总结

本文从 NeuralCDM 模型的结构、原理入手，首先介绍了教育数据挖掘领域常见的成绩预测的方法：认知诊断和协同过滤(矩阵分解)方法，如 DINA、HO-DINA、PMF 等。重点介绍了 AAAI 会议论文(人工智能顶级会议)《Neural Cognitive Diagnosis for Intelligent Education Systems》中的 NeuralCDM 模型，并在公开数据集 ASSIT2009-2010 上进行了复现，此外，还通过爬虫技术爬取了湖南大学 2018 年秋季学期《信号与系统》慕课课程数据，构建了 HNU_SYS 数据集，并在 HNU_SYS 数据集上做了复现。最后，本文基于集成分类的思想，提出了一个融合 NeuralCDM、PMF、HO-DINA 三个模型的成绩预测方法,并在 HNU_SYS 数据集上做了实验验证。